

Spring Data JPA - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SPRING DATA JPA — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: REPOSITORIES	5
1. Introduction	5
1.1 What is it?	5
1.2 Why was it introduced — The EXACT Problem It Solves	5
1.3 How Spring Generates a Working Implementation From JUST an Interface	6
1.4 Interview Definition	6
2. The Repository Interface Hierarchy	6
3. Code Example — A Complete Repository	7
4. Real Life Analogy	7
5. Common Mistakes	7
6. Frequently Asked Interview Questions	7
TOPIC 2: CRUD OPERATIONS	7
1. Introduction	8
2. Code Example — Using Every Core CRUD Method	8
3. Why save() Handles BOTH Insert AND Update	8
4. Optional — Why findById() Returns It, Not the Entity Directly	9
5. Real Life Analogy	9
6. Edge Cases	9
7. Frequently Asked Interview Questions	9
TOPIC 3: DERIVED QUERIES	9
1. Introduction	10
2. Code Examples — Method Name → Generated Query	10
3. The Naming Convention — Key Keywords	11
4. Real Life Analogy	11
5. Edge Cases	11
6. Common Mistakes	11

7. Best Practices	11
8. Frequently Asked Interview Questions	12

TOPIC 4: JPQL 12

1. Introduction	12
2. Code Example — @Query with JPQL	13
3. Modifying Queries — @Modifying	13
4. Real Life Analogy	13
5. Edge Cases	13
6. Common Mistakes	13
7. Frequently Asked Interview Questions	14

TOPIC 5: NATIVE QUERIES 14

1. Introduction	14
2. Code Example	14
3. JPQL vs Native Query — When to Use Which	15
4. Real Life Analogy	15
5. Common Mistakes	15
6. Frequently Asked Interview Questions	15

TOPIC 6: PAGINATION 15

1. Introduction	16
2. Code Example	16
3. What Page Gives You (Beyond Just a List)	16
4. Page vs Slice — A Performance-Conscious Alternative	16
5. Real Life Analogy	16
6. Edge Cases	17
7. Frequently Asked Interview Questions	17

TOPIC 7: SORTING 17

1. Introduction	17
2. Code Examples	17
3. Real Life Analogy	17

4. Common Mistakes	18
5. Frequently Asked Interview Questions	18

TOPIC 8: AUDITING **18**

1. Introduction	18
2. Enabling Auditing	18
3. Code Example — Auditable Fields	19
4. Supplying the "Current User" for @CreatedBy/@LastModifiedBy	19
5. Real Life Analogy	19
6. Common Mistakes	19
7. Frequently Asked Interview Questions	20

TOPIC 9: SPECIFICATIONS **20**

1. Introduction	20
2. Enabling Specifications	20
3. Code Example — Building Dynamic, Composable Filters	20
4. Derived Queries vs JPQL vs Specifications — When to Use Which	21
5. Real Life Analogy	21
6. Common Mistakes	21
7. Frequently Asked Interview Questions	21
8. Revision Notes (Entire Spring Data JPA Section Consolidated)	22
9. Homework (Covers Entire Spring Data JPA Section)	22
FINAL SUMMARY — SPRING DATA JPA COMPLETE	23

SPRING DATA JPA — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: REPOSITORIES

1. Introduction

1.1 What is it?

Spring Data JPA is a layer built ON TOP OF Hibernate/JPA (which you learned in the Hibernate section) that lets you define a DATA ACCESS interface — with ZERO implementation code — and have Spring AUTOMATICALLY generate a fully-working implementation at runtime, handling all the CRUD (Create, Read, Update, Delete) boilerplate for you.

1.2 Why was it introduced — The EXACT Problem It Solves

Recall from the Hibernate section how you had to write a DAO class MANUALLY, even with Hibernate's help:

```
// THE "HIBERNATE ONLY" WORLD - still requires writing a DAO class yourself, method by method
public class EmployeeDao {
    public Employee findById(Long id) {
        try (Session session = sessionFactory.openSession()) {
            return session.get(Employee.class, id);
        }
    }
    public List<Employee> findAll() {
        try (Session session = sessionFactory.openSession()) {
            return session.createQuery("FROM Employee", Employee.class).getResultList();
        }
    }
    public Employee save(Employee emp) {
        try (Session session = sessionFactory.openSession()) {
            Transaction tx = session.beginTransaction();
            session.persist(emp);
            tx.commit();
            return emp;
        }
    }
    public void deleteById(Long id) {
        try (Session session = sessionFactory.openSession()) {
            Transaction tx = session.beginTransaction();
            Employee emp = session.get(Employee.class, id);
            session.remove(emp);
            tx.commit();
        }
    }
    // ... and this ENTIRE pattern REPEATS for EVERY SINGLE entity in your application
    // (Department, Order, Product, Customer...) - dozens of nearly-identical DAO classes!
}
```

With Spring Data JPA, this ENTIRE class collapses into:

```
// THE "SPRING DATA JPA" WORLD - an INTERFACE, with ZERO method bodies written!
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
// THAT'S IT. findById(), findAll(), save(), deleteById() ALL work automatically,
// for EVERY entity, with ZERO implementation code, ever.
}

@Service
public class EmployeeService {
private final EmployeeRepository repository;

public EmployeeService(EmployeeRepository repository) { // Spring INJECTS a fully-working
implementation!
this.repository = repository;
}

public Employee getEmployee(Long id) {
return repository.findById(id).orElseThrow(); // just... works
}
}
```

1.3 How Spring Generates a Working Implementation From JUST an Interface

1. YOU declare: `public interface EmployeeRepository extends JpaRepository<Employee, Long> { }`
|
v
2. Spring Data JPA, AT APPLICATION STARTUP, uses a DYNAMIC PROXY (a runtime-generated class, conceptually similar to Java's Proxy/InvocationHandler mechanism) that IMPLEMENTS this interface, translating EVERY method call into the correct underlying Hibernate/JPA operation automatically.
3. Spring registers this GENERATED implementation as a Spring BEAN, ready to be @Autowired anywhere - EXACTLY as if you'd hand-written the entire DAO class yourself.

1.4 Interview Definition

"Spring Data JPA is a Spring module that dramatically reduces data access boilerplate by allowing developers to define repository interfaces extending `JpaRepository` (or related base interfaces), for which Spring automatically generates a full CRUD implementation at runtime via dynamic proxies, built on top of the underlying JPA/Hibernate persistence layer."

2. The Repository Interface Hierarchy

```
Repository<T, ID> <- marker interface, no methods
|
CrudRepository<T, ID> <- basic CRUD: save(), findById(), findAll(), deleteById(), count()
|
PagingAndSortingRepository<T, ID> <- adds pagination/sorting support (Topic 6)
|
JpaRepository<T, ID> <- adds JPA-specific extras: batch operations, flush(), etc.
(THIS is what you'll extend in virtually all real projects)
```

3. Code Example — A Complete Repository

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository // optional here - JpaRepository sub-interfaces are
public interface EmployeeRepository extends JpaRepository<Employee, Long> { // auto-detected by Spring
    Data anyway
    // Entity type: Employee
    // Primary key type: Long
    // ALL of CrudRepository + JpaRepository's methods are available IMMEDIATELY, for FREE
}
```

4. Real Life Analogy

- **Bank:** Writing a DAO class manually is like a bank teller personally hand-writing every single standard transaction form from scratch, every single time. A Spring Data JPA repository is like the bank having a FULLY AUTOMATED, standardized transaction-processing machine — you just describe WHAT KIND of account you're working with (the interface's type parameters), and the machine handles EVERY standard operation (deposit, withdrawal, balance check) automatically, with zero manual paperwork-writing required.

5. Common Mistakes

- Still writing manual Hibernate Session-based DAO code out of habit, unaware that JpaRepository already provides the SAME functionality with zero implementation effort.
- Not realizing JpaRepository extends PagingAndSortingRepository extends CrudRepository — forgetting which base interface provides which specific methods.

6. Frequently Asked Interview Questions

- 1 **What is Spring Data JPA?** A Spring module that generates fully-working CRUD repository implementations automatically from simple interface declarations, built on top of JPA/Hibernate.
- 2 **What mechanism does Spring Data JPA use to provide an implementation for an interface with no method bodies?** A dynamically-generated runtime PROXY that implements the interface, translating each method call into the correct underlying JPA/Hibernate operation.
- 3 **What's the repository interface hierarchy?** Repository (marker) → CrudRepository (basic CRUD) → PagingAndSortingRepository (adds pagination/sorting) → JpaRepository (adds JPA-specific extras, the one you'll typically extend).
- 4 **What are the two type parameters of JpaRepository<T, ID>?** The ENTITY type (T) and its PRIMARY KEY type (ID).

TOPIC 2: CRUD OPERATIONS

1. Introduction

What is it? This topic covers the actual built-in methods `JpaRepository` gives you for FREE — Create, Read, Update, Delete — that require ZERO code from you beyond declaring the interface.

2. Code Example — Using Every Core CRUD Method

```
@Service
public class EmployeeService {
    private final JpaRepository repository;

    public EmployeeService(JpaRepository repository) { this.repository = repository; }

    public Employee create(Employee emp) {
        return repository.save(emp); // INSERT (if new) - generates the INSERT automatically
    }

    public Employee update(Long id, Employee updatedData) {
        Employee existing = repository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
        existing.setSalary(updatedData.getSalary());
        return repository.save(existing); // UPDATE (existing entity has an ID) - same save() method!
    }

    public Optional<Employee> findOne(Long id) {
        return repository.findById(id); // SELECT by primary key
    }

    public List<Employee> findAll() {
        return repository.findAll(); // SELECT * (all rows)
    }

    public void delete(Long id) {
        repository.deleteById(id); // DELETE
    }

    public long countEmployees() {
        return repository.count(); // SELECT COUNT(*)
    }

    public boolean employeeExists(Long id) {
        return repository.existsById(id); // efficient existence check
    }
}
```

3. Why `save()` Handles BOTH Insert AND Update

```
Employee emp = new Employee();
emp.setName("Alice");
repository.save(emp); // emp.getId() is NULL -> Spring Data JPA treats this as a NEW entity -> INSERT

Employee existing = repository.findById(1L).get();
existing.setSalary(90000.0);
repository.save(existing); // existing.getId() is ALREADY SET -> treated as an EXISTING entity -> UPDATE
```

Internally, `save()` checks whether the entity's ID field is `null` (or whether it exists in the persistence context) to decide whether to call Hibernate's `persist()` (insert) or `merge()` (update) — you never need to call these separately yourself.

4. Optional<T> — Why findById() Returns It, Not the Entity Directly

```
Optional<Employee> maybeEmployee = repository.findById(id);

// SAFE handling - forces you to consider the "not found" case explicitly
Employee emp = maybeEmployee.orElseThrow(() -> new EmployeeNotFoundException(id));

// or, providing a fallback default:
Employee emp2 = maybeEmployee.orElse(new Employee());
```

(This directly ties back to `Optional` from Core Java's Java 8 Features section — `findById()` returning `Optional<Employee>` instead of a potentially-null `Employee` is a deliberate design choice forcing callers to explicitly handle the "not found" case.)

5. Real Life Analogy

- **Library:** `save()` is like a librarian's single, unified "register or update this book's record" desk — if the book has NO existing catalog number, it's registered as brand new; if it ALREADY has one, its existing record is simply updated — the librarian doesn't need two SEPARATE desks for these two scenarios.

6. Edge Cases

Case	Result
Calling <code>save()</code> with an entity whose ID is set to a value that DOESN'T actually exist in the database	Depending on the ID generation strategy, this can either INSERT a new row with that specific ID, or throw an error — behavior varies by generation strategy
Calling <code>deleteById()</code> with an ID that doesn't exist	Throws <code>EmptyResultDataAccessException</code> in most versions
Calling <code>findById()</code> on a non-existent ID	Returns <code>Optional.empty()</code> , NOT null and NOT an exception

7. Frequently Asked Interview Questions

- 1 Does `save()` perform an INSERT or an UPDATE? BOTH — it automatically determines which is appropriate based on whether the entity's ID is already set/exists.
- 2 Why does `findById()` return `Optional<T>` instead of `T` directly? To force callers to explicitly handle the "entity not found" case, rather than risking an unchecked `NullPointerException` from an implicit null.
- 3 What method efficiently checks if an entity exists WITHOUT loading its full data? `existsById()`.

TOPIC 3: DERIVED QUERIES

1. Introduction

What is it? Derived Queries let you define custom queries by simply NAMING a repository method following a specific, structured naming convention — Spring Data JPA PARSES the method name itself and automatically generates the correct query, with STILL ZERO implementation code required.

2. Code Examples — Method Name → Generated Query

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // SELECT e FROM Employee e WHERE e.lastName = ?1
    List<Employee> findByLastName(String lastName);

    // SELECT e FROM Employee e WHERE e.salary > ?1
    List<Employee> findBySalaryGreaterThan(Double salary);

    // SELECT e FROM Employee e WHERE e.department.deptName = ?1 (navigates the RELATIONSHIP!)
    List<Employee> findByDepartmentDeptName(String deptName);

    // SELECT e FROM Employee e WHERE e.firstName = ?1 AND e.lastName = ?2
    List<Employee> findByFirstNameAndLastName(String firstName, String lastName);

    // SELECT e FROM Employee e WHERE e.salary BETWEEN ?1 AND ?2
    List<Employee> findBySalaryBetween(Double min, Double max);

    // SELECT e FROM Employee e WHERE e.lastName LIKE %?1%
    List<Employee> findByLastNameContaining(String partialName);

    // SELECT e FROM Employee e ORDER BY e.salary DESC
    List<Employee> findAllOrderBySalaryDesc();

    // SELECT COUNT(e) FROM Employee e WHERE e.department.deptId = ?1
    long countByDepartmentDeptId(Long deptId);

    // SELECT e FROM Employee e WHERE e.email = ?1 (returns AT MOST one, since email is unique)
    Optional<Employee> findByEmail(String email);

    // DELETE FROM Employee e WHERE e.dept.deptId = ?1
    void deleteByDepartmentDeptId(Long deptId);
}
```

3. The Naming Convention — Key Keywords

Keyword	SQL Equivalent
<code>findBy / getBy / queryBy</code>	<code>SELECT ... WHERE</code>
<code>And / Or</code>	<code>AND / OR</code>
<code>Between</code>	<code>BETWEEN</code>
<code>LessThan / GreaterThan</code>	<code>< / ></code>
<code>Like / Containing</code>	<code>LIKE</code> (Containing wraps the value with <code>%...%</code> automatically)
<code>OrderBy...Asc/Desc</code>	<code>ORDER BY</code>
<code>IgnoreCase</code>	Case-insensitive comparison
<code>In</code>	<code>IN (...)</code>
<code>IsNull / IsNotNull</code>	<code>IS NULL / IS NOT NULL</code>
<code>First3 / Top10</code>	Limits the result count
<code>countBy</code>	<code>SELECT COUNT(...)</code>
<code>deleteBy / removeBy</code>	<code>DELETE ...</code>

4. Real Life Analogy

- **Library:** A derived query is like a librarian who instantly understands a PRECISELY-PHRASED natural-language request ("find all mystery books published after 2010 by this author") and automatically knows EXACTLY which shelves to check and how to combine those criteria — no separate, explicit instructions needed, just a correctly-STRUCTURED request.

5. Edge Cases

Case	Result
A method name with a TYPO in a field name (e.g., <code>findBySalery</code> instead of <code>findBySalary</code>)	FAILS at APPLICATION STARTUP with a clear error — Spring Data JPA validates method names against actual entity fields when the application boots, a genuine "fail fast" safety net
A method name that's too long/complex to express cleanly via naming conventions	Should switch to <code>@Query</code> (JPQL, next topic) instead — derived queries have practical readability limits

6. Common Mistakes

- Writing derived query method names so long and complex they become nearly unreadable — a clear signal to switch to an explicit `@Query` annotation instead.
- Not realizing Spring VALIDATES these method names against your actual entity structure at STARTUP, catching typos immediately rather than at some unpredictable later runtime moment.

7. Best Practices

- Use derived queries for SIMPLE, straightforward filtering conditions.
- Switch to `@Query` (JPQL or native SQL, next topics) once a derived query name would become unwieldy/hard to read.

8. Frequently Asked Interview Questions

- 1 **What is a "derived query" in Spring Data JPA?** A repository method whose query is automatically generated by Spring purely from PARSING its method NAME, following a structured naming convention — no implementation or annotation needed.
 - 2 **What happens if a derived query method name has a typo referencing a non-existent field?** The application FAILS TO START, with a clear error identifying the problem — a deliberate fail-fast safety mechanism.
 - 3 **When should you switch from a derived query to `@Query`?** When the method name would become too long/complex to read cleanly, or when the query logic genuinely can't be expressed via the naming convention alone.
 - 4 **How would you name a method to find employees with a salary greater than a given value, sorted by salary descending?** `List<Employee> findBySalaryGreaterThanOrderBySalaryDesc(Double salary)`.
-

TOPIC 4: JPQL

1. Introduction

What is it? JPQL (Java Persistence Query Language) is the STANDARD, JPA-specification version of Hibernate's HQL (which you learned in the Hibernate section) — an object-oriented query language, used here via Spring Data's `@Query` annotation directly on repository methods, for queries too complex for derived query naming.

(HQL and JPQL are nearly IDENTICAL in syntax and concept — HQL is Hibernate's own, slightly EXTENDED version of the standard JPQL specification. Full query-writing mechanics were covered in the Hibernate section's HQL topic.)

2. Code Example — @Query with JPQL

```
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT e FROM Employee e WHERE e.salary > :minSalary AND e.department.deptName = :deptName")
    List<Employee> findHighEarnersInDepartment(
        @Param("minSalary") Double minSalary,
        @Param("deptName") String deptName);

    @Query("SELECT e.firstName, e.salary FROM Employee e WHERE e.department.deptId = :deptId")
    List<Object[]> findNameAndSalaryByDept(@Param("deptId") Long deptId);

    @Query("SELECT new com.jobsradar.dto.EmployeeSummary(e.firstName, e.salary) " +
        "FROM Employee e WHERE e.department.deptId = :deptId")
    List<EmployeeSummary> findSummariesByDept(@Param("deptId") Long deptId); // DTO projection (clean!)

    @Query("SELECT e FROM Employee e JOIN FETCH e.department WHERE e.salary > :minSalary")
    List<Employee> findHighEarnersWithDept(@Param("minSalary") Double minSalary); // avoids N+1 (Hibernate section!)
}
```

3. Modifying Queries — @Modifying

```
@Modifying // REQUIRED for UPDATE/DELETE queries - marks this as a data-changing operation
@Query("UPDATE Employee e SET e.salary = e.salary * 1.1 WHERE e.department.deptId = :deptId")
int giveDepartmentRaise(@Param("deptId") Long deptId); // returns the NUMBER OF ROWS affected
```

4. Real Life Analogy

- **Library:** JPQL via @Query is like giving the librarian a PRECISE, custom-written request slip when your need is too specific/complex for their standard "quick request phrases" (derived queries) — still phrased in terms of BOOK SUBJECTS/TITLES (entity/field names), not raw physical shelf locations (table/column names).

5. Edge Cases

Case	Result
An @Modifying query WITHOUT the @Modifying annotation itself	Throws an exception — Spring requires this explicit marker for any data-changing (UPDATE/DELETE) @Query
A JPQL query with a typo in an entity/field name	Fails at APPLICATION STARTUP (Spring Data JPA validates @Query JPQL against your entity model eagerly, same fail-fast principle as derived queries)

6. Common Mistakes

- Forgetting @Modifying on an UPDATE/DELETE @Query method.
- Not using @Transactional (typically at the SERVICE layer calling the repository) alongside @Modifying queries — bulk update/delete operations should generally run within an explicit transaction.

7. Frequently Asked Interview Questions

- 1 **What is JPQL, and how does it relate to HQL?** The STANDARD JPA specification's query language; HQL is Hibernate's own implementation, which is a superset/extension of standard JPQL — they're nearly identical in everyday use.
 - 2 **What annotation do you use to write a custom query directly in Spring Data JPA?** `@Query`.
 - 3 **What additional annotation is required for UPDATE/DELETE queries?** `@Modifying`.
 - 4 **How would you avoid the N+1 problem in a custom `@Query`?** Use `JOIN FETCH` to load the related entity eagerly within the same query (exactly the same fix covered in the Hibernate section).
-
-

TOPIC 5: NATIVE QUERIES

1. Introduction

What is it? A native query is RAW SQL (exactly what you learned in the SQL section — real table/column names, not entity/field names) executed directly, via `@Query(nativeQuery = true)` — an escape hatch for the RARE cases where JPQL/derived queries genuinely can't express what you need (vendor-specific SQL functions, highly complex/optimized queries).

2. Code Example

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query(value = "SELECT * FROM employees WHERE salary > :minSalary", nativeQuery = true)
    List<Employee> findHighEarnersNative(@Param("minSalary") Double minSalary);

    @Query(value = "SELECT dept_id, AVG(salary) as avg_salary FROM employees GROUP BY dept_id",
            nativeQuery = true)
    List<Object[]> getAverageSalaryPerDepartment(); // uses a POSTGRESQL-SPECIFIC feature, for example

    @Query(value = "SELECT * FROM employees e WHERE e.attributes @> CAST(:json AS jsonb)", nativeQuery =
            true)
    List<Employee> findByJsonAttribute(@Param("json") String json); // uses PostgreSQL's JSONB @> operator
    // (from the PostgreSQL section!) - something
    // JPQL has NO equivalent for at all
}
```

3. JPQL vs Native Query — When to Use Which

Aspect	JPQL (<code>@Query</code> , default)	Native Query (<code>@NativeQuery = true</code>)
Queries against	Entity/field names	Actual table/column names
Database portability	Portable across different databases (Hibernate translates it)	Tied to your SPECIFIC database's SQL dialect
Access to vendor-specific features	No (e.g., PostgreSQL's JSONB operators, window functions with specific syntax)	Yes — full raw SQL power
Recommendation	DEFAULT choice for custom queries	Use ONLY when JPQL genuinely cannot express what's needed

4. Real Life Analogy

- **Library:** JPQL is like asking the librarian in terms of book subjects/authors (portable across ANY library system). A native query is like walking DIRECTLY to a specific physical shelf yourself, bypassing the librarian's abstraction entirely — powerful and precise, but tied to THIS SPECIFIC library's exact physical layout, and wouldn't work the same way at a different library (database).

5. Common Mistakes

- Overusing native queries by default "just because SQL feels more familiar" — sacrifices database portability and the clean entity-oriented abstraction unnecessarily.
- Forgetting native queries reference actual TABLE/COLUMN names, which can silently drift out of sync with entity mapping changes (`@Column(name=...)` renames) if not kept in mind.

6. Frequently Asked Interview Questions

- 1 **What is a native query in Spring Data JPA?** A raw SQL query (not JPQL) executed via `@Query(nativeQuery = true)`, referencing actual database table/column names directly.
- 2 **When would you reach for a native query instead of JPQL?** When you need database-vendor-specific features (like PostgreSQL's JSONB operators) that JPQL has no equivalent syntax for, or for highly specialized performance-tuned queries.
- 3 **What's the main downside of native queries compared to JPQL?** Loss of database portability — they're tied to your specific database's SQL dialect, unlike JPQL which Hibernate translates appropriately for whichever database is configured.

TOPIC 6: PAGINATION

1. Introduction

What is it? Pagination lets you retrieve LARGE result sets in smaller, manageable CHUNKS ("pages") rather than loading potentially millions of rows into memory at once — essential for any real-world API listing endpoint (e.g., "GET /employees?page=0&size=20").

2. Code Example

```
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Page<Employee> findByDepartmentDeptId(Long deptId, Pageable pageable); // just ADD a Pageable parameter!
}

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @GetMapping
    public Page<Employee> getEmployees(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {

        Pageable pageable = PageRequest.of(page, size);
        return employeeRepository.findAll(pageable); // findAll() ALSO accepts Pageable - built-in!
    }
}
```

3. What `Page<T>` Gives You (Beyond Just a `List`)

```
Page<Employee> resultPage = employeeRepository.findAll(PageRequest.of(0, 20));

resultPage.getContent(); // the ACTUAL List<Employee> for THIS page
resultPage.getTotalElements(); // TOTAL count across ALL pages (e.g., 543 total employees)
resultPage.getTotalPages(); // TOTAL number of pages (e.g., 28 pages of 20 each)
resultPage.getNumber(); // the CURRENT page number
resultPage.hasNext(); // is there a next page?
resultPage.isFirst() / isLast(); // boundary checks
```

Notice `Page<T>` automatically ALSO runs a `COUNT(*)` query behind the scenes to know the total — a genuinely useful, but not "free," extra query.

4. `Page<T>` vs `Slice<T>` — A Performance-Conscious Alternative

```
Slice<Employee> slice = employeeRepository.findByDepartmentDeptId(1L, pageable);
```

`Slice<T>` is like `Page<T>` but does NOT run the extra `COUNT(*)` query — it only knows "IS there a next page?" (by fetching ONE extra row beyond the page size), not the EXACT total count. Use `Slice` when you genuinely don't need the total count (e.g., "infinite scroll" UIs), avoiding that extra count query's cost.

5. Real Life Analogy

- **Library:** Pagination is like a librarian handing you EXACTLY 20 books at a time from a massive collection, rather than dumping the library's ENTIRE million-book inventory in front of you at once — you can request

"the next 20" as needed, and the librarian can ALSO tell you the total book count if asked (the extra COUNT query), or just simply confirm "yes, there are more" without counting everything (the slice alternative).

6. Edge Cases

Case	Result
Requesting a page number BEYOND the actual data (e.g., page 500 when only 10 pages exist)	Returns an EMPTY Page/Slice, not an error
Pageable with a page size of 0 or negative	Throws IllegalArgumentException

7. Frequently Asked Interview Questions

- 1 **Why is pagination important for real-world APIs?** It avoids loading potentially enormous result sets entirely into memory/network transfer at once, which would be slow and resource-intensive.
- 2 **What does Page<T> provide beyond a plain List<T>?** Total element count, total page count, current page number, and navigation helpers (hasNext(), isFirst(), etc.).
- 3 **What's the difference between Page<T> and Slice<T>?** Page runs an EXTRA COUNT(*) query to know the exact total; Slice avoids that cost, only knowing whether a next page exists (useful for infinite-scroll-style UIs where the exact total doesn't matter).

TOPIC 7: SORTING

1. Introduction

What is it? Sorting lets you specify the ORDER in which results are returned — either standalone, or combined directly with pagination.

2. Code Examples

```
import org.springframework.data.domain.Sort;

// Standalone sorting (no pagination)
List<Employee> sorted = employeeRepository.findAll(Sort.by("salary").descending());

// Sorting by MULTIPLE fields
List<Employee> multiSort = employeeRepository.findAll(
    Sort.by("department.deptName").ascending().and(Sort.by("salary").descending()));

// COMBINED with pagination (the most common real-world pattern)
Pageable pageable = PageRequest.of(0, 20, Sort.by("salary").descending());
Page<Employee> page = employeeRepository.findAll(pageable);
```

3. Real Life Analogy

- **Library:** Sorting is like asking the librarian to hand you books ORDERED by publication date, or alphabetically by author — the SAME underlying collection, just presented in a specific, requested order.

4. Common Mistakes

- Sorting by a field name that DOESN'T exist on the entity (a typo) — fails, similarly to derived query name typos, though this is validated at RUNTIME (when the query actually executes) rather than at application startup, since `Sort` objects are typically built dynamically from request parameters.
- Allowing UNVALIDATED, user-supplied sort field names directly into `Sort.by(...)` from a public API — a client could potentially request sorting by a sensitive/internal field not intended for public exposure; validate against an ALLOW-LIST of permitted sort fields.

5. Frequently Asked Interview Questions

- 1 **How do you sort results in Spring Data JPA?** Using `Sort.by("fieldName").ascending().descending()`, passed either standalone to `findAll(Sort)` or combined into a `Pageable` via `PageRequest.of(page, size, sort)`.
- 2 **Can you sort by multiple fields?** Yes — chain multiple `Sort.by(...)` criteria together (e.g., `.and(Sort.by(...))`).
- 3 **Why should you validate user-supplied sort field names against an allow-list in a public API?** To prevent a client from requesting a sort on an internal/sensitive field not intended for public exposure.

TOPIC 8: AUDITING

1. Introduction

What is it? Spring Data JPA's Auditing feature AUTOMATICALLY populates "who created/modified this record, and when" fields on your entities — WITHOUT you manually setting them in every single save/update operation yourself.

2. Enabling Auditing

```
@SpringBootApplication
@EnableJpaAuditing // enables the auditing feature application-wide
public class MyApplication { }
```

3. Code Example — Auditable Fields

```
import org.springframework.data.annotation.*;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@EntityListeners(AuditingEntityListener.class) // hooks Hibernate's lifecycle events into Spring's auditing
public class Employee {
    @Id @GeneratedValue private Long id;
    private String firstName;

    @CreatedDate
    @Column(updatable = false) // NEVER changed after initial creation
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;
}
```

4. Supplying the "Current User" for @CreatedBy/@LastModifiedBy

```
@Configuration
public class AuditingConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> {
            // In a REAL app, this would read the CURRENTLY AUTHENTICATED user
            // (from Spring Security's context, covered in the Spring Security section)
            Authentication auth = SecurityContextHolder.getContext().getAuthentication();
            return Optional.ofNullable(auth != null ? auth.getName() : "system");
        };
    }
}
```

5. Real Life Analogy

- **Bank:** Auditing is like every bank transaction AUTOMATICALLY stamping itself with "processed by Teller #4, at 2:35 PM" — the SYSTEM records this metadata automatically on every action, without the teller needing to manually fill in a separate log entry for every single transaction they process.

6. Common Mistakes

- Forgetting @EnableJpaAuditing entirely — the @CreatedDate/@LastModifiedDate annotations simply do NOTHING without it.
- Forgetting to provide an AuditorAware bean when using @CreatedBy/@LastModifiedBy — these fields require Spring to know HOW to determine "the current user," which isn't automatic.

7. Frequently Asked Interview Questions

- 1 **What does Spring Data JPA's auditing feature do?** Automatically populates "created/modified by whom, and when" fields on entities, without manual code in every save/update operation.
- 2 **What annotation enables auditing application-wide?** `@EnableJpaAuditing`.
- 3 **What four field-level annotations does auditing provide?** `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`.
- 4 **What must you provide for `@CreatedBy/@LastModifiedBy` to actually work?** An `AuditorAware<T>` bean telling Spring how to determine the "current user" for each operation.

TOPIC 9: SPECIFICATIONS

1. Introduction

What is it? The `Specification<T>` API lets you build TYPE-SAFE, DYNAMICALLY COMPOSABLE query conditions — Spring Data JPA's integration of the JPA Criteria API (which you learned in the Hibernate section) directly into the repository pattern, ideal for complex, OPTIONAL, filter-driven search endpoints.

2. Enabling Specifications

```
public interface EmployeeRepository extends JpaRepository<Employee, Long>,
JpaSpecificationExecutor<Employee> { // adds Specification support
}
```

3. Code Example — Building Dynamic, Composable Filters

```
import org.springframework.data.jpa.domain.Specification;

public class EmployeeSpecifications {

    public static Specification<Employee> hasSalaryGreaterThan(Double minSalary) {
        return (root, query, cb) -> cb.greaterThan(root.get("salary"), minSalary);
    }

    public static Specification<Employee> inDepartment(String deptName) {
        return (root, query, cb) -> cb.equal(root.join("department").get("deptName"), deptName);
    }

    public static Specification<Employee> hasLastName(String lastName) {
        return (root, query, cb) -> cb.equal(root.get("lastName"), lastName);
    }
}

@Service
public class EmployeeSearchService {
    private final EmployeeRepository repository;
}
```

```

public List<Employee> search(Double minSalary, String deptName, String lastName) {
    Specification<Employee> spec = Specification.where(null); // start with an "always true" base

    if (minSalary != null) {
        spec = spec.and(EmployeeSpecifications.hasSalaryGreaterThan(minSalary));
    }
    if (deptName != null) {
        spec = spec.and(EmployeeSpecifications.inDepartment(deptName));
    }
    if (lastName != null) {
        spec = spec.and(EmployeeSpecifications.hasLastName(lastName));
    }

    return repository.findAll(spec); // combines ONLY the conditions actually provided!
}
}

```

This EXACT pattern — conditionally adding `.and(...)` clauses ONLY when a filter value is actually present — is genuinely awkward to express cleanly with plain JPQL string-building, but is CLEAN and TYPE-SAFE with Specifications, echoing the SAME dynamic-query-building benefit the Criteria API provided directly in Hibernate.

4. Derived Queries vs JPQL vs Specifications — When to Use Which

Approach	Best For
Derived Queries	Simple, FIXED filter conditions, known in advance
JPQL (@Query)	More complex but still FIXED, well-known queries
Specifications	DYNAMIC, OPTIONAL, conditionally-combined filter criteria (e.g., a flexible "advanced search" feature)

5. Real Life Analogy

- **Library:** Specifications are like a librarian who can COMBINE any number of OPTIONAL search criteria a reader provides ("mystery genre" AND "published after 2015" AND "available in paperback") — but SKIPS any criteria the reader didn't actually specify, cleanly composing only the ACTUALLY relevant filters together, rather than needing a separate, pre-written search method for every possible COMBINATION of criteria.

6. Common Mistakes

- Trying to build the SAME dynamic, optional-filter logic using string-concatenated JPQL — far messier and more error-prone than the clean, composable `Specification` approach.
- Forgetting to have the repository extend `JpaSpecificationExecutor<T>` — `findAll(Specification)` isn't available without it.

7. Frequently Asked Interview Questions

- 1 **What is a `Specification<T>` in Spring Data JPA?** A type-safe, composable way to build dynamic query conditions, based on the JPA Criteria API, integrated directly into the repository pattern.
- 2 **What interface must a repository extend to support Specifications?** `JpaSpecificationExecutor<T>`.
- 3 **When are Specifications the best choice over derived queries or JPQL?** When building queries with MANY OPTIONAL, dynamically-combined filter criteria (like an advanced search feature), where the exact combination of conditions isn't known in advance.

8. Revision Notes (Entire Spring Data JPA Section Consolidated)

SPRING DATA JPA – CHEAT SHEET

=====

SOLVES: manual Hibernate DAO boilerplate - JUST declare an interface, Spring generates a full implementation via a runtime proxy

HIERARCHY: Repository -> CrudRepository (basic CRUD) -> PagingAndSortingRepository
-> JpaRepository (extend THIS in practice)

CRUD: save() handles BOTH insert/update (checks if ID exists), findById() returns Optional<T> (forces explicit "not found" handling)

DERIVED QUERIES: method NAME is parsed into a query automatically
findByFieldName, findByXAndY, findByFieldGreaterThan, findByFieldContaining, etc.
TYPO in field name -> FAILS AT STARTUP (fail-fast validation)

JPQL (@Query): entity/field-name based custom queries, @Modifying required for UPDATE/DELETE, JOIN FETCH avoids N+1 (same as Hibernate)

NATIVE QUERY (@Query(nativeQuery=true)): raw SQL, table/column names, loses portability, use ONLY for vendor-specific features JPQL can't express

PAGINATION: Pageable/PageRequest.of(page, size) -> Page<T> (has getTotalElements(), extra COUNT query) vs Slice<T> (no count query, just hasNext())

SORTING: Sort.by("field").ascending()/.descending(), combine with Pageable

AUDITING: @EnableJpaAuditing + @CreatedDate/@LastModifiedDate/@CreatedBy/
@LastModifiedBy - needs an AuditorAware bean for the "By" fields

SPECIFICATIONS: JpaSpecificationExecutor<T> + Specification<T> - type-safe, DYNAMICALLY composable filters, best for optional/advanced-search-style queries

9. Homework (Covers Entire Spring Data JPA Section)

- 1 Convert a manually-written Hibernate session-based DAO class (from the Hibernate section) into a one-line JpaRepository interface, and verify identical CRUD behavior.
- 2 Write 3 derived query methods (a simple filter, a combined And filter, and an OrderBy variant) against your Employee entity.
- 3 Add pagination AND sorting to a REST endpoint listing employees, testing different page/size/sort query parameters.
- 4 Enable JPA auditing on your Employee entity and verify createdAt/updatedAt populate automatically on save/update.
- 5 Build a Specification-based dynamic search endpoint accepting optional minSalary, department, and lastName query parameters.

FINAL SUMMARY — SPRING DATA JPA COMPLETE

This concludes the **Spring Data JPA** section — 9 topics:

- 1 **Repositories** — the exact manual-DAO boilerplate it eliminates, the dynamic proxy mechanism.
- 2 **CRUD Operations** — `save()`'s dual insert/update behavior, `Optional<T>` from `findById()`.
- 3 **Derived Queries** — method-name-based automatic query generation, fail-fast validation.
- 4 **JPQL** — `@Query` for custom entity-oriented queries, `@Modifying`, DTO projections.
- 5 **Native Queries** — raw SQL escape hatch for vendor-specific features.
- 6 **Pagination** — `Page<T>` vs `Slice<T>`.
- 7 **Sorting** — standalone and combined with pagination.
- 8 **Auditing** — automatic created/modified metadata.
- 9 **Specifications** — type-safe, dynamically composable queries for advanced search.

This is the final piece of the persistence-layer puzzle — Core Java → JDBC → SQL → PostgreSQL → Hibernate → Spring Data JPA, each layer building on the one before it. Next: Spring Security, covering how to actually protect these APIs.

(END OF SPRING DATA JPA MASTER NOTES — ready to proceed to Spring Security whenever you say "Next Topic.")